



What is Playwright ?

Playwright is an open-source end-to-end (E2E) testing framework developed by Microsoft. It allows you to automate web browsers such as Chrome, Firefox, Safari, and Edge using a single API

You can use Playwright to:

- Test web applications automatically.
- Simulate user actions (clicks, typing, navigation).
- Capture screenshots and videos.
- Handle multiple browser contexts.

It supports

- **JavaScript**
- **TypeScript**
- **Python**
- **Java**
- **.NET**



Why Use Playwright?

- **Cross-browser support:** Chromium, WebKit, Firefox.
- **Reliable testing:** Waits automatically for elements to be ready.
- **Headless & headed modes:** Run tests with or without UI.
- **Fast execution:** Parallel test execution.
- **Record/replay tests:** Generate tests from user actions.
- Built-in assertions and test runner.

Installation

To use Playwright with Node.js, follow these steps:

📌 Prerequisites:

Install [Node.js](https://nodejs.org) (v14+ recommended)

► Download:

- Go to: <https://nodejs.org>
- Click “LTS” version and install it.

► Check if installed:

- Open your terminal (Command Prompt or PowerShell) and run:

```
C:\Users\User>npm -v
10.7.0

C:\Users\User>node -v
v20.15.0

C:\Users\User>
```

STEP 1: Create a Project Folder

Open terminal and run:

```
mkdir playwright-demo
```

```
cd playwright-demo
```

```
Microsoft Windows [Version 10.0.26100.4484]
(c) Microsoft Corporation. All rights reserved.

D:\SLT QA Reports>mkdir playwright-demo

D:\SLT QA Reports>cd playwright-demo

D:\SLT QA Reports\playwright-demo>|
```

STEP 2: Install Playwright

Run this command: `npm init playwright@latest`

```
Microsoft Windows [Version 10.0.26100.4484]
(c) Microsoft Corporation. All rights reserved.

D:\SLT QA Reports>mkdir playwright-demo

D:\SLT QA Reports>cd playwright-demo

D:\SLT QA Reports\playwright-demo>npm init playwright@latest

> npx
> create-playwright

Getting started with writing end-to-end tests with Playwright:
Initializing project in '.'
? Do you want to use TypeScript or JavaScript? ...
> TypeScript
  JavaScript
```

► **During setup:**

It will ask:

Install Playwright browsers → say Yes

Add GitHub Actions workflow → Optional

Choose JavaScript or TypeScript → Choose JavaScript for now

```

Create playwright
Getting started with writing end-to-end tests with Playwright:
Initializing project in '.'
/ Do you want to use TypeScript or JavaScript? · JavaScript
/ Where to put your end-to-end tests? · tests
/ Add a GitHub Actions workflow? (y/N) · false
/ Install Playwright browsers (can be done manually via 'npx playwright install')? (Y/n)
· true

Initializing NPM project (npm init -y)-
Wrote to D:\SLT QA Reports\playwright-demo\package.json:

{
  "name": "playwright-demo",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

Installing Playwright Test (npm install --save-dev @playwright/test)...

```

After the Setup

It will:

Add the default example tests in tests/example.spec.js. &

Option 1 : write script in that file

OR

Option 2: You can Automatically generate Playwright test code by clicking through your app manually.

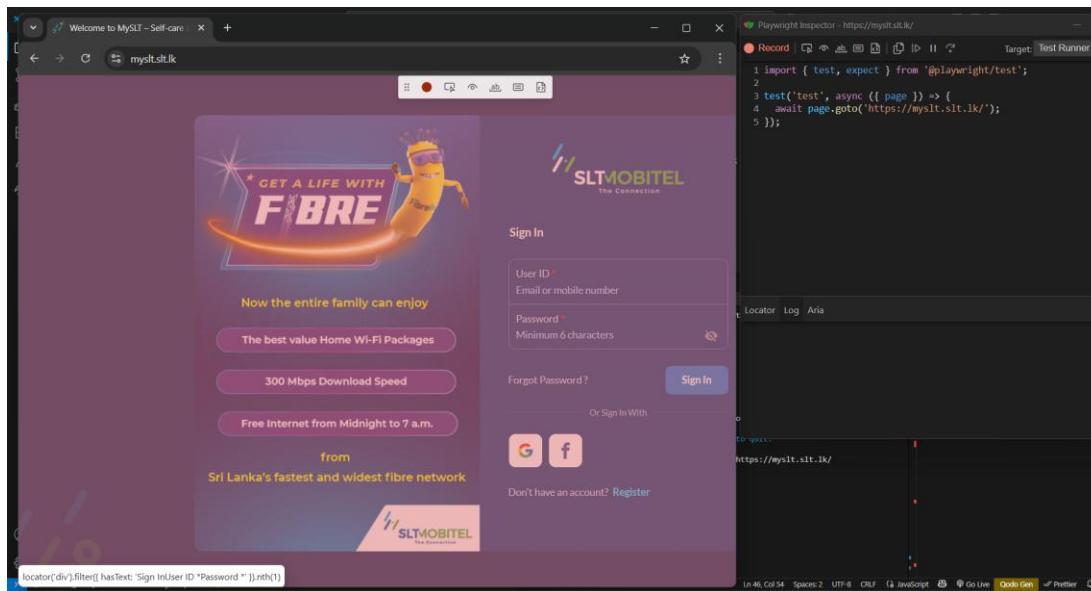
1. Open your terminal and run:
npx playwright codegen <https://myslt.slt.lk/>

```

Terminate batch job (Y/N)? y
❖PS D:\SLT QA Reports\playwright-demo> npx playwright codegen https://myslt.slt.lk/
>>
|

```

This will open a **browser window** and start recording your interactions.



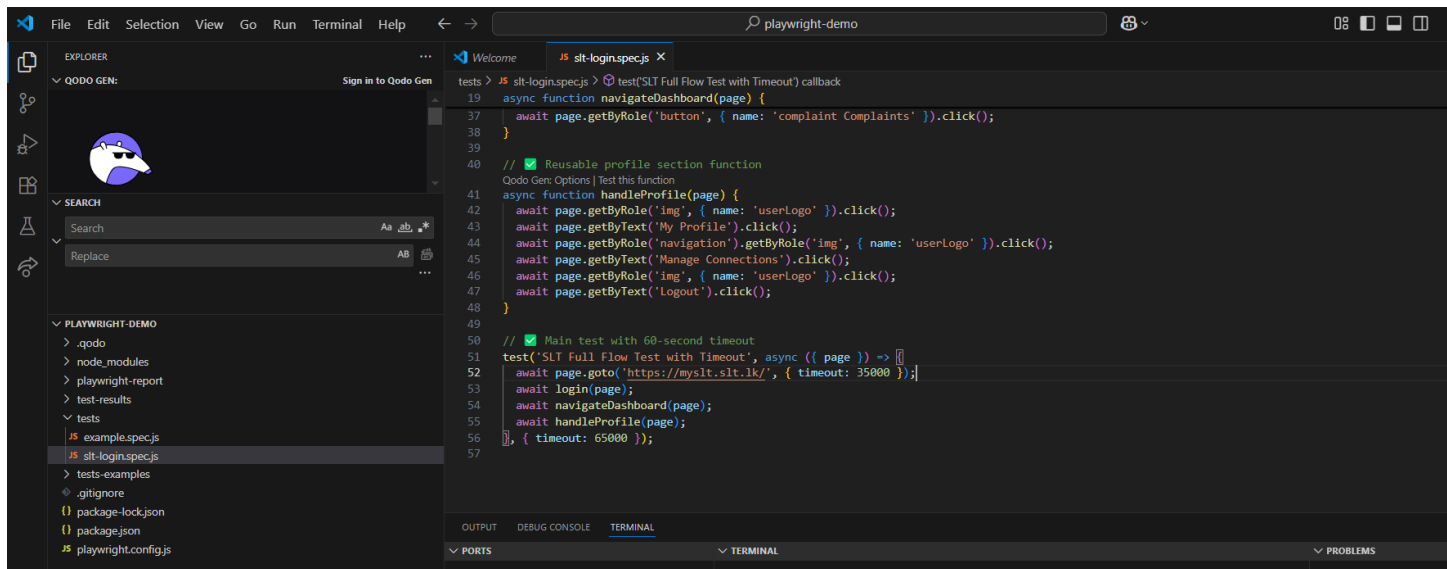
2. Perform your actions manually in the browser:

- Fill in the login fields
- Click buttons
- Navigate to pages

As you interact, Playwright records all the steps and shows you the test code **in real time** in the terminal or separate codegen window.

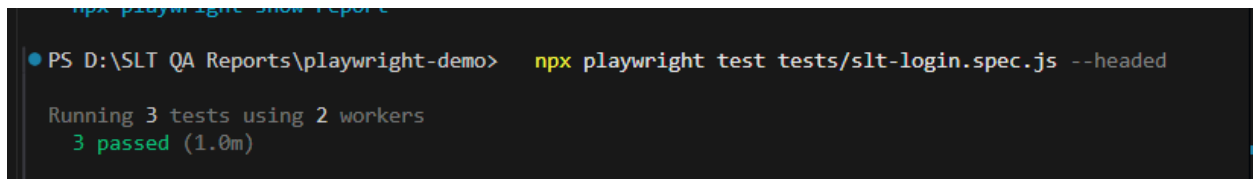
3. Save the generated code:

You can copy the code and paste it into a file like: tests/slt-login.spec.js

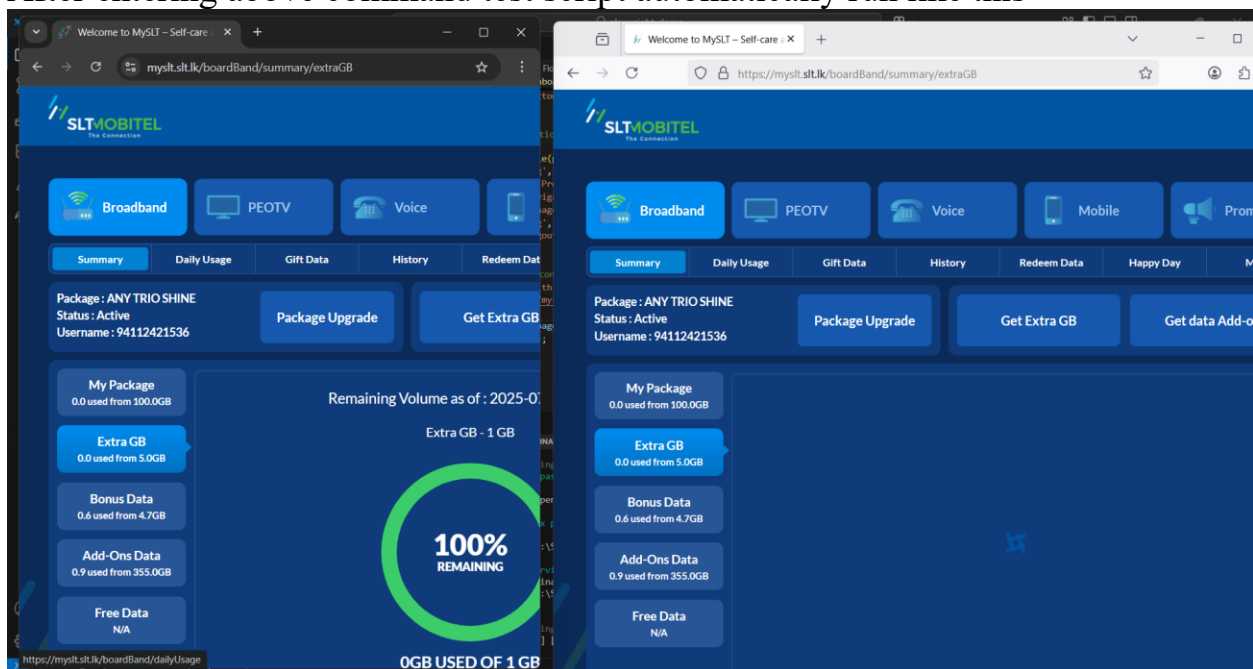


Then run your own test again:

`npx playwright test tests/slt-login.spec.js --headed`



After entering above command test script automatically run like this



Why use --headed?

So instead of running the browser in the background (headless), it will open a real visible browser window.

Reason	Explanation
Visual debugging	You can see what's happening step by step useful if a test fails or clicks go to the wrong place.
UI behavior	You want to watch how the login or navigation works , especially for dynamic apps like SLT.
Troubleshooting	Easier to identify issues like modals, popups, element overlap, or incorrect element selection.
Development phase	While building your tests, it's better to run in headed mode to fine-tune selectors, timing, and interactions.

If you want to get test report give this command :

```
npx playwright show-report
```

```
npx playwright show-report
npx playwright show-report
>>> D:\SLT QA Reports\playwright-demo>
Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```

Report will display like this :

Q		All 3	Passed 3	Failed 0	Flaky 0	Skipped 0
7/7/2025, 9:32:46 AM Total time: 1.0m						
v slt-login.spec.js						
✓	SLT Full Flow Test with Timeout	chromium	24.8s			
slt-login.spec.js:51						
✓	SLT Full Flow Test with Timeout	firefox	29.4s			
slt-login.spec.js:51						
✓	SLT Full Flow Test with Timeout	webkit	26.9s			
slt-login.spec.js:51						

Q p:chromium

All 3Passed 3Failed 0Flaky 0Skipped 0

SLT Full Flow Test with Timeout

slt-login.spec.js:5218.6s

chromium

✓ Run

Test Steps

> ✓ Before Hooks867ms

> ✓ Navigate to "/" — slt-login.spec.js:58.8s

> ✓ Wait for selector getByRole("textbox", { name: 'Email or mobile number' }) — slt-login.spec.js:1069ms

> ✓ Fill "qatestslt001@gmail.com" getByRole("textbox", { name: 'Email or mobile number' }) — slt-login.spec.js:1161ms

> ✓ Wait for selector getByRole("textbox", { name: 'Minimum 6 characters' }) — slt-login.spec.js:1317ms

> ✓ Fill "TestQA@1234" getByRole("textbox", { name: 'Minimum 6 characters' }) — slt-login.spec.js:1435ms

> ✓ Click getByRole("button", { name: 'Sign In' }) — slt-login.spec.js:16120ms

> ✓ Click getByRole("link", { name: '/Extra GB/i' }) — slt-login.spec.js:215.7s

> ✓ Click getByRole("link", { name: '/Bonus Data/i' }) — slt-login.spec.js:22106ms

> ✓ Click getByRole("link", { name: '/Add-Ons Data/i' }) — slt-login.spec.js:2388ms

> ✓ Click getByRole("link", { name: '/Free Data/i' }) — slt-login.spec.js:24100ms

> ✓ Click getByRole("button", { name: '/Package Upgrade/i' }) — slt-login.spec.js:2583ms

> ✓ Click getByRole("button", { name: '/PEOTV/i' }) — slt-login.spec.js:26121ms

> ✓ Click getByRole("button", { name: '/Voice/i' }) — slt-login.spec.js:27105ms